55. KubeRay: Run Ray on Kubernetes (with Examples)

Builds on **#50 (Docker)** and **#53 (Kubernetes)**.

> **In one line: Ray** runs heavy Python/AI work across many machines. **KubeRay** is the Kubernetes operator that **creates, scales and manages Ray clusters automatically** from a YAML file.

1. First: what is Ray?

Ray is an open-source Python framework for running work in parallel on many machines, mainly for **AI, ML and data processing**.

> **Real-life example: exam paper checking** 📝
> One teacher checking 1,000 papers takes 10 days. Split them among 20 teachers and it takes half a day.
> Ray is the **head examiner**: it splits the work (papers) among workers (machines), collects the results, and reassigns work if a teacher falls sick.

```
import ray
ray.init()

@ray.remote                 # ← "this function can run on any machine in the cluster"
def process(image):
    return resize(image)

futures = [process.remote(img) for img in images]   # starts all in parallel 🚀
results = ray.get(futures)                          # wait and collect results
```

What people use Ray for

Ray library	Use	Example
Ray Core	Run any Python function in parallel	Process 1M images
Ray Data	Big data processing	Clean 500 GB of logs
Ray Train	Train ML models on many GPUs	Train a PyTorch model on 8 GPUs
Ray Tune	Try many model settings in parallel	Test 100 learning rates
Ray Serve	Serve ML models as an HTTP API	`POST /predict` → sentiment result

History: why Ray was introduced

- Built at **UC Berkeley's RISELab** (around 2017), the same lab family that created Apache Spark.
- **Problem:** AI workloads (training, simulations, serving) needed to scale beyond one machine, but existing tools were either too rigid (Spark, built for batch data) or too low-level (writing your own distributed code).
- **Solution:** Ray lets you write **normal Python**, add `@ray.remote`, and it runs across a cluster.
- The creators founded **Anyscale** to support it. Ray is used by companies like OpenAI, Uber, Spotify and Shopify.

2. What is KubeRay, and why was it needed?

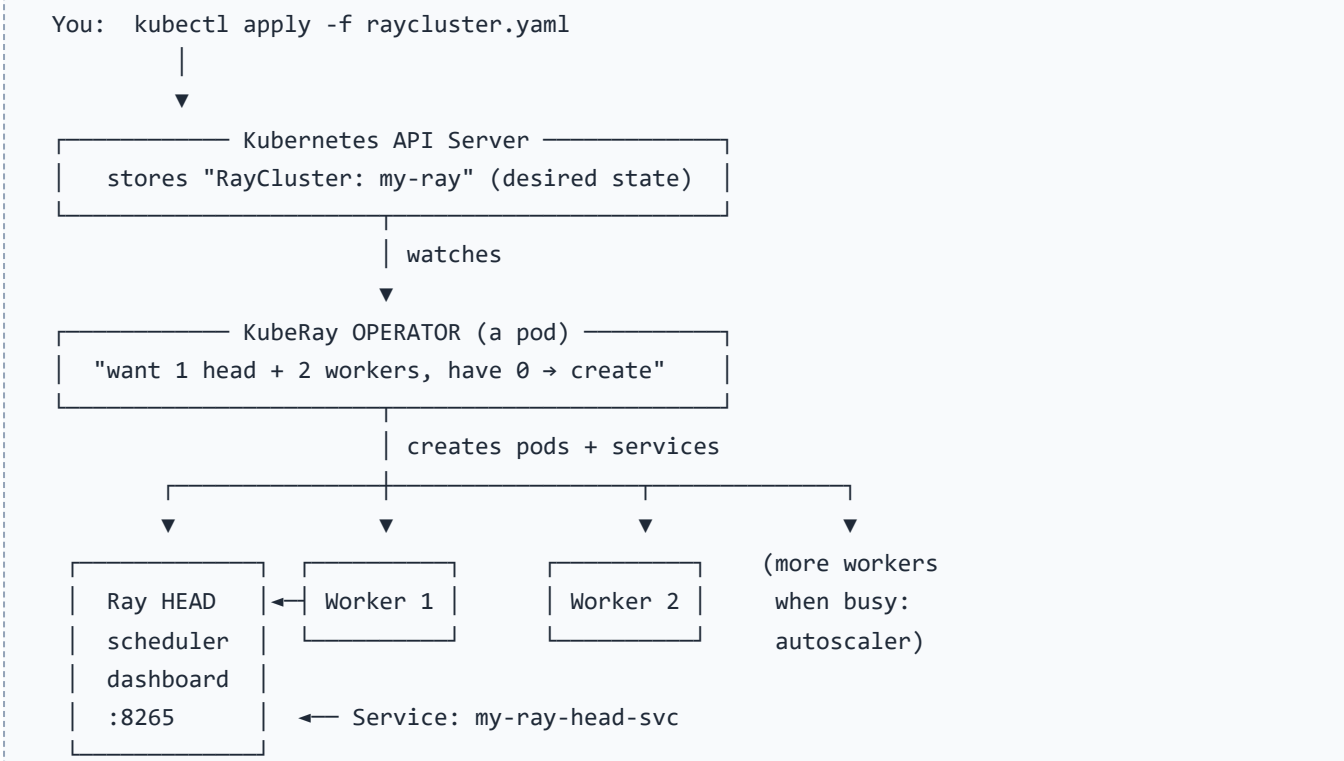
A Ray cluster has **1 head node** (the boss: scheduler + dashboard) and **many worker nodes** (they do the work).

Without KubeRay	With KubeRay
Start the head and workers on each machine by hand	Write **one YAML**, and the cluster is created automatically
A worker crashes and stays dead until you fix it	Self-healing: the pod is recreated
Add or remove machines by hand	**Autoscaling**: workers are added when there's work and removed when idle
Forget to shut down and keep paying for GPUs 💸	**RayJob** deletes the cluster when the job finishes
Upgrade the model API with downtime	**RayService**: zero-downtime upgrades

KubeRay is an open-source **Kubernetes operator** (started by the community, including Ant Group and others, and now part of the official `ray-project`). Companies already ran everything on Kubernetes and wanted Ray there too, managed the "Kubernetes way".

> **What is an "operator"?** A program running inside Kubernetes that knows how to manage **one specific kind of software**. It adds new resource types (like `RayCluster`) to Kubernetes and keeps them in the state you asked for, using the same **desired-state loop** from theory #53.
> **Example:** Kubernetes is a general hospital manager, and KubeRay is a **specialist doctor** who only handles Ray patients.

3. How KubeRay works (architecture)



1. You apply YAML describing the Ray cluster you want.
2. Kubernetes stores it. The **KubeRay operator** sees the new `RayCluster`.
3. The operator creates a **head pod**, **worker pods** and a **Service** for the head.
4. Workers connect to the head, and Ray is ready.
5. When a lot of work arrives, the **Ray autoscaler** asks for more workers, and KubeRay creates more pods (up to `maxReplicas`). When they're idle, they're removed.
6. A pod crashes? The operator recreates it ♻️

4. KubeRay's 3 resources

Resource	What it does	Lifetime	Use it for
RayCluster	A Ray cluster (head + workers)	Runs until you delete it	Experiments, notebooks, shared team cluster
RayJob	Creates a cluster → runs your script → (optionally) deletes the cluster	Only while the job runs	Training, batch processing, nightly jobs
RayService	A RayCluster + a **Ray Serve** app as an HTTP API, with zero-downtime upgrades	Long-running	Serving ML models in production

> **Restaurant example: RayCluster** = a kitchen that's always open. **RayJob** = a caterer hired for one event, who leaves after. **RayService** = a takeaway counter (API) that's always serving customers, and you can change the menu without closing.

5. Setup (step by step)

You need

Tool	Why
A Kubernetes cluster	Locally: Docker Desktop K8s / kind / minikube. Cloud: EKS / GKE / AKS.
`kubectl`	To talk to Kubernetes
`helm`	The Kubernetes package manager (like npm for K8s apps)
(optional) `pip install "ray[default]"`	For the `ray job submit` command on your PC

Step 1: Install the KubeRay operator

```
helm repo add kuberay https://ray-project.github.io/kuberay-helm/
helm repo update
helm install kuberay-operator kuberay/kuberay-operator

kubectl get pods
```

```
# kuberay-operator-7d8f9c-xxxx   1/1   Running ✅

kubectl get crd | grep ray
# rayclusters.ray.io, rayjobs.ray.io, rayservices.ray.io   ← new resource types
```

> Check the KubeRay docs for the latest operator version and pin it in production: `helm install kuberay-operator kuberay/kuberay-operator --version <x.y.z>`.

6. Example 1: RayCluster

raycluster.yaml

```yaml
apiVersion: ray.io/v1
kind: RayCluster
metadata:
  name: my-ray
spec:
  enableInTreeAutoscaling: true          # ← let Ray add/remove workers
  headGroupSpec:
    rayStartParams: {}
    template:
      spec:
        containers:
          - name: ray-head
            image: rayproject/ray:2.9.0  # use a current Ray version
            resources:
              requests: { cpu: "500m", memory: "1Gi" }
              limits:   { cpu: "1",    memory: "2Gi" }
            ports:
              - { containerPort: 6379, name: gcs }        # Ray internal
              - { containerPort: 8265, name: dashboard }  # web dashboard
              - { containerPort: 10001, name: client }    # Ray client
  workerGroupSpecs:
    - groupName: cpu-workers
      replicas: 1
      minReplicas: 0                     # scale to zero when idle 💰
      maxReplicas: 3                     # never more than 3
      rayStartParams: {}
      template:
        spec:
          containers:
            - name: ray-worker
              image: rayproject/ray:2.9.0
              resources:
                requests: { cpu: "500m", memory: "1Gi" }
                limits:   { cpu: "1",    memory: "2Gi" }
```

> Port **6379** here is Ray's own internal control store (GCS), which uses the same port number as Redis by tradition. It's **not** your app's Redis cache from theory #49.

Create and check it

```
kubectl apply -f raycluster.yaml

kubectl get rayclusters
# NAME     DESIRED WORKERS   AVAILABLE WORKERS   STATUS
# my-ray   1                 1                   ready

kubectl get pods
# my-ray-head-xxxxx                    Running
# my-ray-cpu-workers-worker-xxxxx      Running

kubectl get svc
# my-ray-head-svc   ClusterIP   ...   6379, 8265, 10001
```

Open the Ray dashboard

```
kubectl port-forward svc/my-ray-head-svc 8265:8265
# open http://localhost:8265 → nodes, jobs, CPU/memory, logs
```

Run Python code on the cluster

```
# square.py (on your PC)
import ray
ray.init()

@ray.remote
def square(x):
    return x * x

print(ray.get([square.remote(i) for i in range(10)]))
```

```
# submit it (port-forward from above must be running)
ray job submit --address http://localhost:8265 --working-dir . -- python square.py
# → [0, 1, 4, 9, 16, 25, 36, 49, 64, 81] ✅
```

The 10 `square` calls ran **in parallel** across the head and worker pods.

7. Example 2: RayJob (run once, then clean up)

Good for training or batch jobs. You don't leave a cluster running and paying.

rayjob.yaml

```
apiVersion: ray.io/v1
kind: RayJob
metadata:
  name: count-primes
spec:
  entrypoint: |
    python -c "
    import ray
    ray.init()

    @ray.remote
    def count_primes(start, end):
        def is_prime(n):
            return n > 1 and all(n % d for d in range(2, int(n**0.5) + 1))
        return sum(1 for n in range(start, end) if is_prime(n))

    chunks = [(i, i + 100_000) for i in range(0, 1_000_000, 100_000)]
    total = sum(ray.get([count_primes.remote(a, b) for a, b in chunks]))
    print('Primes below 1,000,000:', total)
    "
  shutdownAfterJobFinishes: true         # ← delete the cluster when done 💰
  ttlSecondsAfterFinished: 60            # keep it 60 s so you can read logs
  rayClusterSpec:
    headGroupSpec:
      rayStartParams: {}
      template:
        spec:
          containers:
            - name: ray-head
              image: rayproject/ray:2.9.0
              resources:
                limits: { cpu: "1", memory: "2Gi" }
    workerGroupSpecs:
      - groupName: workers
        replicas: 2
        minReplicas: 2
        maxReplicas: 2
        rayStartParams: {}
        template:
          spec:
            containers:
              - name: ray-worker
                image: rayproject/ray:2.9.0
                resources:
                  limits: { cpu: "1", memory: "2Gi" }
```

```
kubectl apply -f rayjob.yaml
kubectl get rayjobs                       # JOB STATUS: RUNNING → SUCCEEDED
kubectl logs -l job-name=count-primes     # → Primes below 1,000,000: 78498
```

```
apply ──► cluster created ──► job runs (10 chunks in parallel) ──► SUCCEEDED
                                                                       │
                                                  60 s later ──► cluster deleted 🗑
```

For real projects, put your code in a Docker image (or a `runtimeEnvYAML` with `working_dir`) instead of inline Python.

8. Example 3: RayService (serve an ML model as an API)

The model code: sentiment.py (Ray Serve)

```
from ray import serve
from starlette.requests import Request

@serve.deployment(num_replicas=2)          # 2 copies for load + safety
class Sentiment:
    def __init__(self):
        from transformers import pipeline
        self.model = pipeline("sentiment-analysis")   # loads once per replica

    async def __call__(self, request: Request):
        body = await request.json()
        result = self.model(body["text"])[0]
        return {"label": result["label"], "score": round(result["score"], 3)}

app = Sentiment.bind()
```

rayservice.yaml

```
apiVersion: ray.io/v1
kind: RayService
metadata:
  name: sentiment
spec:
  serveConfigV2: |
    applications:
      - name: sentiment-app
        import_path: sentiment:app            # file:variable
        route_prefix: /predict
        runtime_env:
          working_dir: "https://github.com/<you>/<repo>/archive/<commit>.zip"
          pip: ["transformers", "torch"]
  rayClusterConfig:
    headGroupSpec:
      rayStartParams: {}
      template:
        spec:
          containers:
            - name: ray-head
              image: rayproject/ray:2.9.0
```

```
            resources: { limits: { cpu: "2", memory: "4Gi" } }
            ports:
              - { containerPort: 8000, name: serve }   # HTTP API
    workerGroupSpecs:
      - groupName: workers
        replicas: 1
        minReplicas: 1
        maxReplicas: 4
        rayStartParams: {}
        template:
          spec:
            containers:
              - name: ray-worker
                image: rayproject/ray:2.9.0
                resources: { limits: { cpu: "2", memory: "4Gi" } }
```

```
kubectl apply -f rayservice.yaml
kubectl get rayservices                  # wait for status: Running / ready
kubectl get svc                          # sentiment-serve-svc   port 8000

kubectl port-forward svc/sentiment-serve-svc 8000:8000
curl -X POST http://localhost:8000/predict \
     -H "Content-Type: application/json" \
     -d '{"text": "This course is amazing!"}'
# → {"label": "POSITIVE", "score": 0.999}
```

> **Zero-downtime upgrade:** change the code (a new `working_dir` zip) and run `kubectl apply` again. KubeRay starts the new version, waits until it's healthy, switches traffic, and only then removes the old one, the same idea as a rolling update in #53.

9. Connect it to your NestJS app

Ray is **Python** and your API is **NestJS**. They talk over **HTTP**, and NestJS never needs to know about Ray.

```
  Angular ──► NestJS API (users, auth, DB, Redis cache)
                  │
                  │  POST http://sentiment-serve-svc:8000/predict
                  ▼
            Ray Serve on KubeRay (ML model, heavy compute, autoscaling)
```

ml.service.ts

```
import { Injectable, ServiceUnavailableException } from '@nestjs/common';

@Injectable()
export class MlService {
  private readonly url = process.env.RAY_SERVE_URL;
```

```
  // in-cluster:  http://sentiment-serve-svc:8000
  // local test:  http://localhost:8000  (with port-forward)

  async sentiment(text: string) {
    const res = await fetch(`${this.url}/predict`, {
      method: 'POST',
      headers: { 'Content-Type': 'application/json' },
      body: JSON.stringify({ text }),
      signal: AbortSignal.timeout(10_000),       // don't hang forever
    });
    if (!res.ok) throw new ServiceUnavailableException('ML service failed');
    return res.json();                           // { label, score }
  }
}
```

reviews.controller.ts

```
@Post('analyze')
analyze(@Body('text') text: string) {
  return this.mlService.sentiment(text);
}
```

> **Combine with Redis (#49 / #54):** cache the results as `sentiment:<hash of text>` with a TTL. Repeated texts then skip the expensive model call.

10. Useful commands

```
kubectl get rayclusters,rayjobs,rayservices    # all Ray resources
kubectl describe raycluster my-ray             # details + events
kubectl get pods -l ray.io/cluster=my-ray      # pods of one cluster
kubectl logs <head-pod>                         # head logs
kubectl port-forward svc/my-ray-head-svc 8265:8265   # dashboard

ray job submit --address http://localhost:8265 -- python script.py
ray job list   --address http://localhost:8265
ray job logs   --address http://localhost:8265 <job-id>

kubectl delete raycluster my-ray               # remove cluster
helm uninstall kuberay-operator                # remove operator
```

11. GPUs (for real ML)

```
workerGroupSpecs:
  - groupName: gpu-workers
    minReplicas: 0                  # no GPU cost when idle 💰
    maxReplicas: 4
    rayStartParams: {}
```

```
    template:
      spec:
        containers:
          - name: ray-worker
            image: rayproject/ray-ml:2.9.0-gpu
            resources:
              limits: { nvidia.com/gpu: 1, cpu: "4", memory: "16Gi" }
```

In Python: `@ray.remote(num_gpus=1)`. Ray sends that task only to GPU workers, and KubeRay starts a GPU pod if none is running.

12. Common problems

Problem	Why	Fix
Pods stuck in `Pending`	Not enough CPU/RAM in the cluster	Lower the resource requests, or add nodes
`OOMKilled`	The pod used more memory than its limit	Raise the `memory` limit (ML models need GBs)
Image pull takes forever	Ray images are big (GBs)	Wait for the first pull, or use a smaller image tag
Job can't import your module	Code isn't in the container	Use `--working-dir`, `runtime_env`, or bake it into the image
Version mismatch errors	The Ray version on your PC ≠ the cluster's	Use the same Ray version everywhere
RayService never becomes ready	The model download is slow, or there's a bug in the app	`kubectl describe rayservice` and the head pod logs

13. Your laptop (8 GB RAM)

A Ray head + worker needs roughly **2–4 GB RAM**, plus Kubernetes itself. On your 8 GB laptop (already ~73% used), a local KubeRay setup will be **very tight**.

Recommended path:

1. Learn Ray first **without Kubernetes**: `pip install "ray[default]"`, then run `square.py` locally.
2. Then try KubeRay with **1 small head + 0–1 worker** (lower the limits to 1Gi), closing Chrome and other apps first.
3. For real use, use a cloud cluster (EKS/GKE) and delete it after practice.

14. Do you need KubeRay?

Use KubeRay if...	You DON'T need it if...
You train or serve ML models at scale	You build normal CRUD APIs (NestJS + Postgres)
You process big data in parallel with Python	Your work fits on one machine
Your company already runs Kubernetes	You're still learning Docker/K8s basics
You need GPUs that scale to zero when idle	A simple managed AI API (e.g. a hosted model endpoint) is enough

Alternatives: plain Ray on VMs, **Anyscale** (managed Ray), AWS SageMaker / Google Vertex AI (managed ML), or Celery/BullMQ for simple background jobs.

15. Quick summary

Question	Answer
What is Ray?	A Python framework to run AI/data work in parallel across many machines
Who made Ray?	UC Berkeley RISELab (~2017), now backed by Anyscale
What is KubeRay?	A Kubernetes **operator** that creates and manages Ray clusters from YAML
Why KubeRay?	Automatic setup, self-healing, autoscaling, cleanup and zero-downtime upgrades on Kubernetes
Install	`helm install kuberay-operator kuberay/kuberay-operator`
3 resources	**RayCluster** (long-running), **RayJob** (run once + cleanup), **RayService** (model API)
Ray parts	1 **head** (scheduler + dashboard :8265) + many **workers**
Run code	`ray job submit --address http://localhost:8265 -- python file.py`
Serve API port	`<name>-serve-svc:8000`
With NestJS	NestJS calls Ray Serve over HTTP; cache the results in Redis
Learning order	Docker → Kubernetes → Ray locally → KubeRay